

EEE3096S Lecture 19 Activity : Computer Architecture



The Mission:

Design Simple Peripheral Controller Processor (PCP)

Introduction:

In this activity, you will delve into the world of computer architecture and ISA (Instruction Set Architecture) design by creating a simplified peripheral controller, which we are giving the code-name PCP ... so as not to confuse this with the more sophisticated PIC (Peripheral Interface Controller) family of microcontrollers that have been around for a long time. The PCP controller will be responsible for forwarding input data from one source to a destination, demonstrating a fundamental concept in computer systems.

Scenario:

Imagine a scenario where you need to read data from an 8-bit parallel port and transmit it serially to another device. You'll design a processor with a minimal instruction set to accomplish this task efficiently. The processor can be considered as awesomely fast, like nanosecond propagation delays (i.e. GHz clock). It runs in parallel to larger, more complex host processor(s) on the computer mainboard. The PCP is mainly intended for transferring data from one source, such as an ADC connected to a microphone input and moves this data to another point such as an input buffer that the host processor reads.

Sample Program:

Refer to the provided sample program (Listing 1) to understand the desired functionality of the PCP. This program showcases how the processor should handle input/output operations, bit manipulation, and control flow. The visualization, see Figure 1, indicates how the PCP is integrated into a larger computer system to facilitate data exchange and provide an aspect of instruction-based reconfigurability (i.e. the PCP program) to a system.

```
// Set up ports
SETIN  #0x10      // Set port 0x10 as an input port (assume ports are tristate)
SETOUT #0x20      // Set port 0x20 as an output port

MAINLOOP:
    WAITIN          // wait for an input to arrive on an input line
    IN      A, #0x10 // Get input from port 0x10 into register A

    // now stream out the parallel input as sequence of serial bits in low endian
    SET      B, #128 // set bit B to 128, for checking and sending the MSB first

LOOP:
    AND      A, B      // check if n'th bit of A is high or low, going from bit 7
                        // to bit 0
    OUTZ     #0, #0x20 // if bit checked is zero then output constant value #0 to
                        // port 0x20
    OUTNZ    #1, #0x20 // if bit is not zero output constant value #1 to port 0x20
    SHR      B         // non-rotational shift right one bit of B register
    JNZ      #LOOP     // if B is not zero then look back to label LOOP

    // just continuously loops the program from label MAINLOOP
    J        #MAINLOOP // jump back to start of MAINLOOP
```

Listing 1: Converting data received on 8-bit input port sent out serially to serial line.

This program probably takes 11 instructions, one instruction for each assembly operation statement. This would perhaps take $6\text{ns} \times 8 \text{ loops} + 3\text{ns} +$ for one iteration of the whole program, thus 51ns for one iteration of the whole program. That is pretty fast, and assuming a read of the input line takes just 1ns . So, the max transfer speed it can do ...

$$51 \text{ ns} = 51 * 10^{-9} \text{ seconds} = 5.1 * 10^{-8} \text{ seconds}$$

$$\text{Time to transfer one byte} = 5.1 * 10^{-8} \text{ seconds}$$

$$\text{Transmission speed} = 8 \text{ bits} / (5.1 * 10^{-8} \text{ seconds}) = 156,862,745 \text{ bps} = 156.86 \text{ Mbps (Megabits/s)}$$

So, that can't really handled USB2 (i.e. 480 Mbps) but can handle USB1.1 (12Mbps).

If we had it managing more than just that one transfer, the max speed would drop further.

Tasks for you to do:

1. Instruction Set Definition:

- Carefully analyze the sample program and identify the essential instructions required for its execution.
- Create a comprehensive list of these instructions, considering their operands, addressing modes, and potential side effects.
- Brainstorm additional instructions that might enhance the processor's capabilities or simplify programming.

2. Instruction Encoding:

- Decide on a suitable instruction encoding scheme, determining the bit patterns representing each instruction and its operands.
- Strive for an efficient encoding that minimizes instruction size while maintaining clarity and ease of decoding.

3. Processor Block Diagram:

- Develop a more refined high-level block diagram illustrating the major components of your PCP processor, such as its:
 - ALU (Arithmetic Logic Unit)
 - Registers (and its Register Bank)
 - Control Unit
 - Instruction Decoder
 - Memory Interface
 - I/O Interface

- Show the data flow between these components and how instructions are fetched, decoded, and executed.

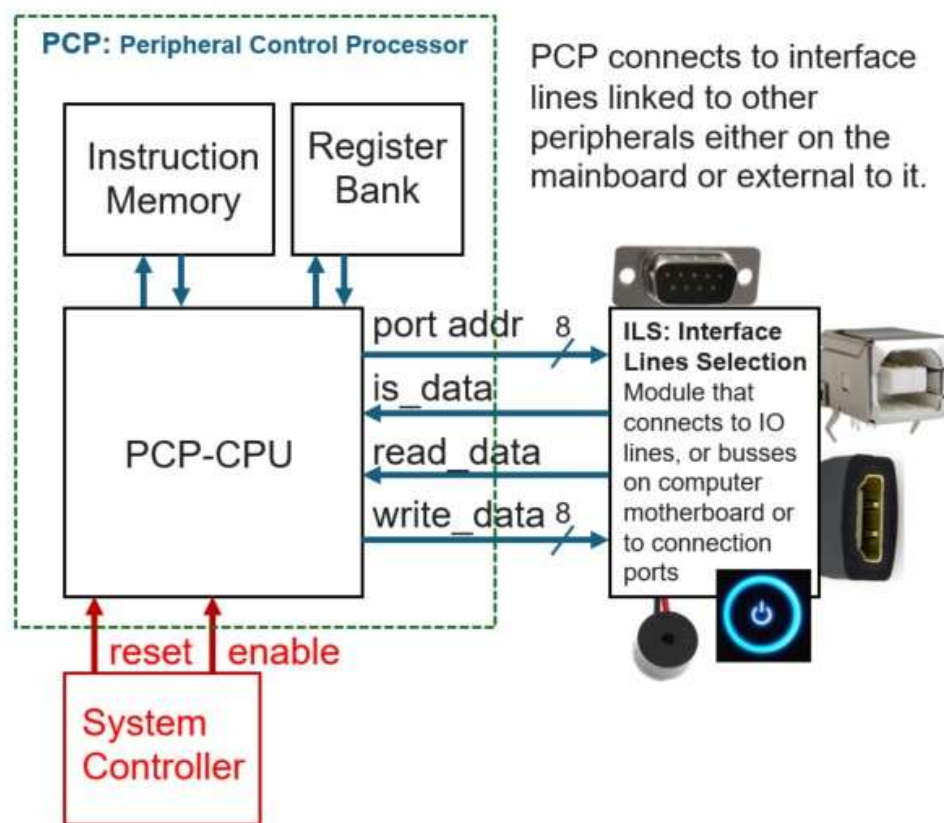
4. Mind Mapping/Connections:

- Utilize mind mapping or other visual techniques to explore the relationships between instructions, data flow, and processor components.
- This exercise can help solidify your understanding of the processor's internal workings and potential design trade-offs.

5. Partial Design Presentation (just have a go at aspects, not whole thing!):

- Prepare a concise presentation summarizing your processor design, including:
 - Instruction set overview
 - Instruction encoding scheme
 - Processor block diagram
 - Key design decisions and justifications

Visualization:



Block diagram showing how PCP is integrated into larger computer system.

Figure 1: simplified block diagram of the PCP (the PCP-CPU part would include the ALU, control unit, etc.). The arrows indicate data flow between components. The ILS is a separate subsystem that connects to actual peripheral IO lines.

As the diagram indicates, the PCP is essentially itself a peripheral that is connected to a host processor. You could consider it something like a DMAC (Direct Memory Access Controller)... but a DMAC is a more complex, if still very task-specific type of processor which focuses on transferring data to or from memory (e.g. the computers main RAM memory); the host processor still 'talks' to the DMAC to tell it what to do. The PCP, in contrast, does not 'talk' to its host processor, it merely shifts data from one place to the other (although doing so, it may be communicating in a rather rudimentary way with the host computer, for example raising interrupt, dispatching data when interrupt acknowledge; but not being able to do anything more complicated, as a DMAC can do, such as having the host processor tell the DMAC to transfer data from one address to another).

Conclusion:

The aim is that by completing this activity, you will gain valuable insights into the intricacies of computer architecture and ISA design, albeit it provided to just a simple processor. You will learn to translate high-level program requirements into concrete hardware implementations, laying a strong foundation for further exploration in this exciting field. This PCP is in reality a type of processing element sometimes needed in real designs, for instance in FPGA solutions where you may need to capture data from possibly many different sources, possibly at the same time, and forward the data on in a suitably structured manner to a host processor or pc.

Remember, the focus here is on understanding the core concepts and design process, not on intricate low-level details. Have fun designing your peripheral controller processor!
